

園遊會收銀與訂單流程管理系統

詹秉睿 清水高中

特殊選才專案報告

讓各組透過共同的訂單資訊協作

擔任班級園遊會總召時，我將點餐、分組製作與櫃台交付整合成網站。
收銀送單後，各組查看自己的工作，櫃台則依整張訂單的完成狀態交付。
我以三週完成開發，沿用家中設備部署，並在活動現場協調系統與人員運作。

2026.04.18 活動使用 201 筆營業期間訂單

需求與開發條件

需要處理的工作

攤位同時販售涼麵、糖葫蘆與飲料。客製化品項需要傳給不同製作組，櫃台也需要知道整張訂單是否備齊，才能核對並交付。

我設定的目標

收銀專注點餐，各製作組專注自己的品項，櫃台掌握交付狀態。希望減少跨組傳話與反覆詢問，讓工作進度直接呈現在訂單中。

約三週開發

1 位管理員、3 位員工

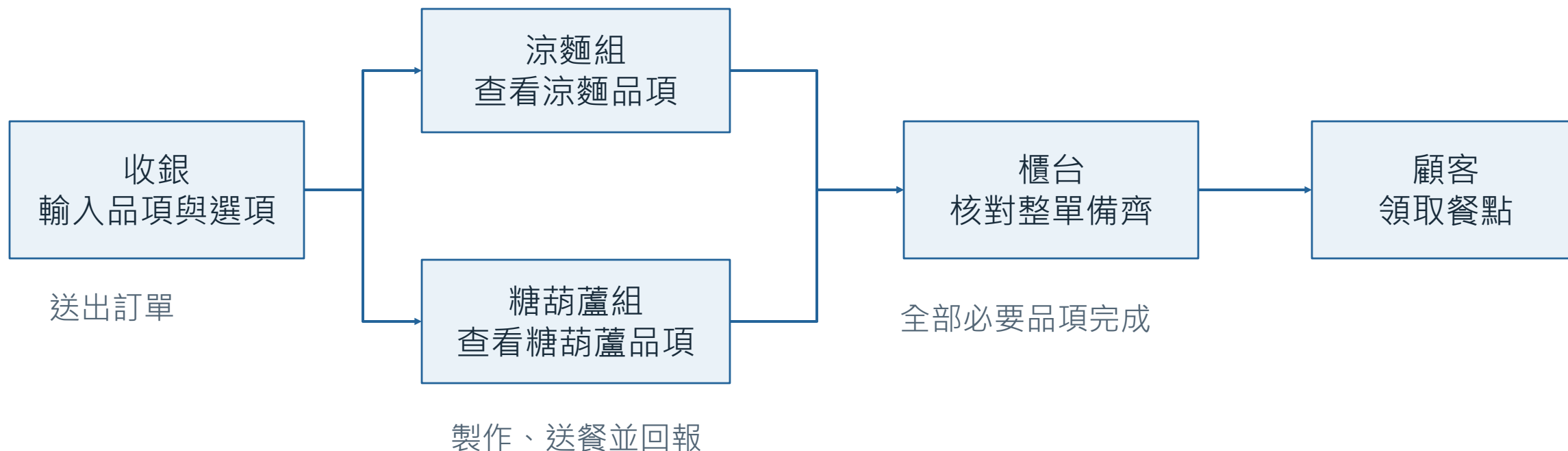
服務 1-2 天

活動營業 08:45–15:00。管理員與員工使用自己的行動數據，顧客亦可查單。

使用規模：當天另有動態進出的顧客約 10 位使用查詢功能，此描述不作為訪客統計。

工作分流與訂單流程

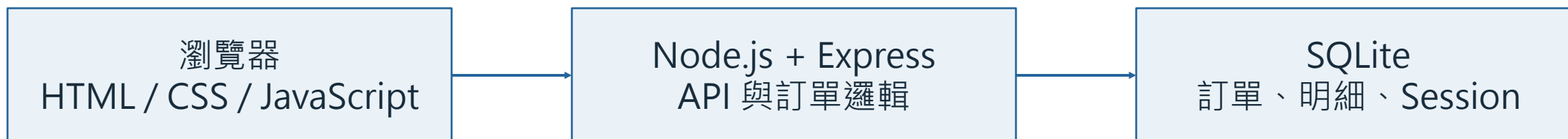
同一單號串接不同單位，各組回報自己的完成狀態。



管理員在後台查看訂單與時段資料，支援各組，並處理取消及誤送訂單。

本圖呈現正常操作流程。飲料不設製作組旗標，純飲料訂單可直接進入可取餐狀態。

系統架構與選擇理由



原生前端

處理表單與畫面更新，
減少前端建置步驟。
以 HTTP 輪詢取得狀態。

Express 後端

處理路由、計價與狀態。
前端提交品項與選項，
後端重新計算金額。

SQLite 儲存

資料庫隨應用程式部署，
不另架資料庫服務。
配合本次短期使用需求。

我向 AI 諮詢後，以減少除錯與部署複雜度作為選擇依據。

登入使用隨機 Token 與 SQLite Session。角色頁面負責工作分流，並非完整權限隔離的證明。

家中伺服器的對外部署

服務只需一至兩天，我沿用家中電腦與網路，避免增加主機租用費用。



這次架設讓我把網域解析、網路入口、代理與應用程式的職責連在一起理解。

程式與 Nginx 範例使用 127.0.0.1:3000 作為後端；不據此推定活動當天的外部連接埠。

混合訂單的完成條件

一張同時包含涼麵與糖葫蘆的訂單，需要兩組都完成，才能通知櫃台備齊。

涼麵組	糖葫蘆組	整張訂單
未完成	未完成	等待 / 製作中
已完成	未完成	製作中
已完成	已完成	可取餐

ramenDone = 不需要涼麵 || 涼麵已完成
hawsDone = 不需要糖葫蘆 || 糖葫蘆已完成
ready = ramenDone && hawsDone

依原始碼簡化的判斷邏輯。無該類品項時，不必等待對應製作組。

資料保存與製作管理

建單資料一起保存

訂單主檔保存單號、狀態與總額，明細保存品項、數量及選項。

建單交易包住明細寫入、糖葫蘆預製數量扣減與初始日誌，讓這些操作一併提交。

管理對象	現有做法
涼麵	回報售罄，控制是否繼續接單。
糖葫蘆	依口味記錄預製數量。所有所需口味足夠時一起扣減，並標記糖葫蘆組完成；不足則進入待製作流程。

此處管理預製品與售罄狀態，不涵蓋完整原料庫存；建單交易也不代表所有例外操作都具完整一致性。

同一筆訂單在各工作頁的呈現

示範單 #1：涼麵與綜合糖葫蘆。三張畫面分別取自送單、製作與備齊階段。

收銀：確認整張訂單



涼麵組：只顯示涼麵



櫃台：確認品項備齊



以同一單號連接作業，但每個頁面只呈現該階段需要的資訊與操作。

本頁為實際系統的示範操作，非活動現場照片。

基本功能驗證

以獨立資料庫操作正常訂單流程，核對 API 回應與頁面顯示。

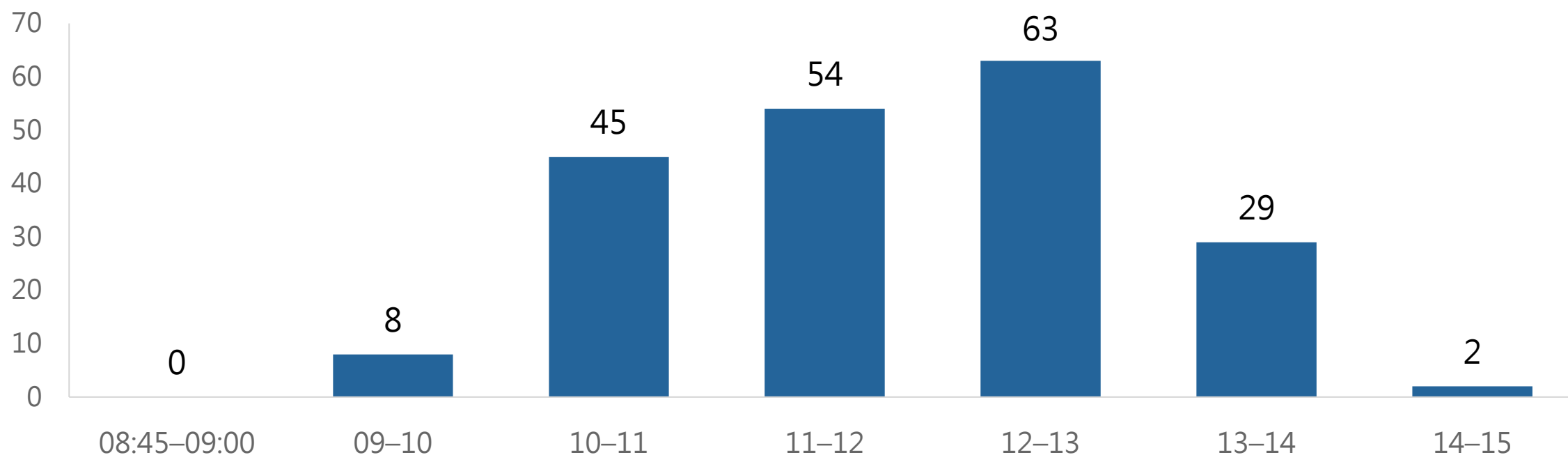
驗證項目	觀察結果	結果
建單與計價	混合單 85 元、兩項明細；純紅茶 15 元。	通過
製作組分流	相同單號，各組取得自己的品項。	通過
整單完成條件	一組完成仍製作中，兩組完成才可取餐。	通過
退回重做	重設指定組旗標，另一組完成狀態保留。	通過
交付與查詢	櫃台標記領取後，顧客查詢顯示已領取。	通過

混合單使用自備餐具選項。此為基本功能檢查，未涵蓋壓力測試、資安驗證或現場效率比較。

營業期間每小時建單量

201 筆訂單

最高峰 12:00–13:00，共 63 筆



訂單集中在 10:00–13:00，共 162 筆，占營業期間建單量約 80.6%。

來源：系統訂單資料庫。首段為 08:45–09:00；包含後續退款需求，不代表顧客人數或實收營收。

餐點也需要保留訂單識別

現場發生的落差

網站已把訂單分給各製作組，但送到櫃台的食物沒有標示單號。
多筆餐點混在一起時，櫃台仍需要重新協調餐點與訂單的對應。

當天的應對

我作為管理員協調各組支援櫃台，並協助各筆訂單的配餐與交付。
這讓我看到：畫面上的完成狀態，需要能對應到現場拿到的那份食物。

改善構想：製作組送出餐點時附上單號，讓櫃台沿用同一識別核對。

餐點標識為活動後反思提出的改善構想，尚未實作。

取消與退款的作業分工

現場處理	後續設計方向
取消與誤送由管理員協助處理， 需在支援各組時另外介入。	讓收銀端處理日常取消與誤送， 並保存原因及處理紀錄。
未交餐等需退款的訂單已現場退款， 資料庫未同步完成退款登錄。	將退款與訂單結案串接， 使財務資料反映最終處理結果。

我希望把一般例外交回最接近需求的工作單位，
讓管理員能保留時間處理跨組協調。

以上皆為改善構想。現有訂單資料可分析建單需求，金額則不能直接視為實收或淨利。

個人貢獻與學習

需求與架構由我主導

此系統由我獨立主導架構與開發，我也擔任班級園遊會總召。
我從工作分工出發規劃功能與架構，完成部署，並在活動現場維護及協調。

AI 協助工具選擇與程式開發

開發過程使用 AI 協作，我向 AI 詢問工具選擇與程式做法。
我以短期服務的需求判斷方案，優先降低除錯與部署複雜度。

部署方面，我理解了對外服務需要網域、網路入口與應用程式共同配合。
流程方面，我學到資訊狀態、實體餐點與例外處理也必須相互對應。

資料來源與版本

專案與活動資料

Cashier-System 原始碼

基準 commit : 3263e7e , 另含工作樹修改

核心依據 : server.js 、 schema.sql 、 前端程式

活動資料 : 2026.04.18 , 08:45–15:00

來源 : cashier.db 與 WAL , 共 201 筆建單

本人訪談 : 需求、分工、現場經驗與反思

技術文件

Express : Hello world example
expressjs.com/en/starter/hello-world/

SQLite : Appropriate Uses For SQLite
sqlite.org/whentouse.html

Nginx : HTTP proxy module
nginx.org/en/docs/http/

DuckDNS : Why Duck DNS
duckdns.org/why.jsp

驗證環境 : Node.js v24.18.0 、 Express 4.22.2 、 better-sqlite3 11.10.0 ; 隔離資料庫。

基本功能驗證 : 2026-09-10 。 本報告未以測試結果回推活動當天的效能。

寫作參考 : 《專題報告撰寫、審查重點與完整範本研究報告》。官方文件查閱日 : 2026-09-10 。